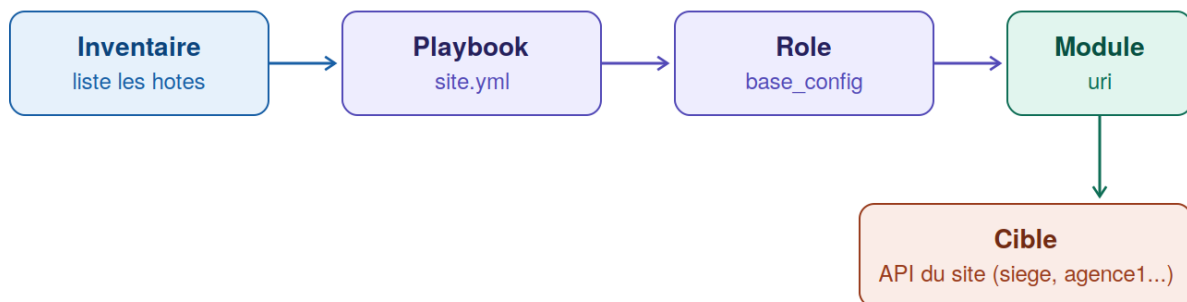


TP5 — Créer une architecture Ansible modulaire avec rôles et inventory dynamique

Formation NetOps : Automatiser la gestion de son Infrastructure Réseau — Ambient IT

Description



Chaque tâche d'un rôle appelle un module. Le module est le seul à parler à la cible.

Figure 1: Flux d'exécution Ansible

Ce TP structure une architecture Ansible modulaire (rôle, inventaire dynamique) qui pilote les 3 sites NexaCorp du lab Docker (TP4) via leur API REST. Plutôt qu'un inventaire statique en dur, un script Python génère l'inventaire à partir d'un fichier `sites.yml` — ce script pourra être étendu plus tard pour se connecter à NetBox ou à un vrai CMDB.

Prérequis

- TP1 à TP4 validés, lab Docker du TP4 démarrable (`docker compose up`)
- Ansible fonctionnel (voir Étape 2 — sous Windows, Ansible ne tourne pas nativement et nécessite WSL)

Étapes

Étape 1 — Créer sa branche pour ce TP

```
cd netops-project-<nom-groupe>
git checkout dev
```

```
git pull origin dev
git checkout -b <prenom>-<nom>-jour2-tp5
```

Étape 2 — Installer Ansible

Sous Windows : Ansible ne fonctionne pas nativement, il faut passer par **WSL**.

Dans PowerShell, en administrateur :
wsl --install -d Ubuntu

Redémarrez si demandé, puis cherchez **“Ubuntu”** dans le menu Démarrer (une icône dédiée, distincte de “Docker Desktop”) et lancez-le. Créez un utilisateur/mot de passe Linux à la première ouverture.

Vérifiez que vous êtes bien dans Ubuntu et pas ailleurs :

```
cat /etc/os-release
```

Ça doit afficher PRETTY_NAME="Ubuntu ...". Si ça affiche PRETTY_NAME="Docker Desktop", vous êtes dans le terminal interne de Docker (sa VM de maintenance) — ce n'est pas le bon terminal, ouvrez “Ubuntu” depuis le menu Démarrer à la place.

Installer Ansible (dans le terminal Ubuntu) — utilisez apt, pas pip, pour éviter l'erreur externally-managed-environment :

```
sudo apt update
sudo apt install -y ansible
```

Retrouver votre projet depuis WSL : vos fichiers créés avec Git Bash sont sur le disque Windows, accessibles depuis WSL via /mnt/c/... :

```
cd /mnt/c/Users/<votre-nom>/Desktop/netops-project
```

Astuce pour y arriver directement à chaque ouverture du terminal :

```
echo 'cd /mnt/c/Users/<votre-nom>/Desktop/netops-project' >> ~/.bashrc
```

Activer Docker depuis WSL : Docker Desktop tourne sur Windows (installé au TP4) ; il faut l'exposer à WSL. Dans Docker Desktop : **Settings** > **Resources** > **WSL integration** → activer le toggle **Ubuntu** → **Apply & Restart**. Vérifiez ensuite dans le terminal Ubuntu :

```
docker --version
docker compose version
```

macOS :

```
brew install ansible
```

Linux (natif) :

```
sudo apt update
sudo apt install -y ansible
```

Vérifier :

```
ansible --version
```

Étape 3 — Créer le fichier source de vérité des sites

```
cat > inventory/sites.yml << 'EOF'
sites:
  - nom: siege
    role: hub
    api_port: 5001
  - nom: agence1
    role: spoke
    api_port: 5002
  - nom: agence2
    role: spoke
    api_port: 5003
EOF
```

Étape 4 — Écrire l'inventaire dynamique

```
cat > inventory/dynamic_inventory.py << 'EOF'
#!/usr/bin/env python3
"""
Inventaire dynamique NetOps NexaCorp.
Lit inventory/sites.yml et genere un inventaire Ansible JSON.
"""

import json
import os
import sys
import yaml

HERE = os.path.dirname(os.path.abspath(__file__))
SITES_FILE = os.path.join(HERE, "sites.yml")

def build_inventory():
    with open(SITES_FILE) as f:
        data = yaml.safe_load(f)

    inventory = {
        "_meta": {"hostvars": {}},
        "all": {"children": ["hub", "spokes"]},
        "hub": {"hosts": []},
        "spokes": {"hosts": []},
    }

    for site in data["sites"]:
        name = site["nom"]
        group = "hub" if site["role"] == "hub" else "spokes"
```

```

        inventory[group]["hosts"].append(name)
        inventory["_meta"]["hostvars"][name] = {
            "ansible_connection": "local",
            "ansible_python_interpreter": sys.executable,
            "api_url": f"http://localhost:{site['api_port']}",
            "site_role": site["role"],
        }

    return inventory

def main():
    if len(sys.argv) > 1 and sys.argv[1] == "--list":
        print(json.dumps(build_inventory()))
    elif len(sys.argv) > 2 and sys.argv[1] == "--host":
        print(json.dumps({}))
    else:
        print("Usage: --list ou --host <nom>", file=sys.stderr)
        sys.exit(1)

if __name__ == "__main__":
    main()
EOF
chmod +x inventory/dynamic_inventory.py

```

ansible_connection: local : on ne se connecte pas en SSH à ces sites — ce sont des conteneurs Docker exposant une API REST, pas des machines Linux classiques. Ansible exécute donc les modules directement en local, et c'est le module uri (HTTP) qui parle réellement au site distant, comme pour DNA Center/ACI/SD-WAN plus tard dans la formation. Sur un vrai routeur Cisco, on utiliserait plutôt ansible_connection: network_cli ou httpapi.

Étape 5 — Tester l'inventaire seul

```
ansible-inventory -i inventory/dynamic_inventory.py --graph
```

Résultat attendu :

```

@all:
  |--@ungrouped:
  |--@hub:
  |   |--siege
  |--@spokes:
  |   |--agence1
  |   |--agence2

```

Étape 6 — Créer le rôle base_config

```
mkdir -p roles/base_config/tasks roles/base_config/defaults
```

```
cat > roles/base_config/defaults/main.yml << 'EOF'
```

```
---
```

```
device_id_prefix: "router"
```

```
EOF
```

```
cat > roles/base_config/tasks/main.yml << 'EOF'
```

```
---
```

- name: Verifier que le site repond (health check)
 ansible.builtin.uri:
 url: "{{ api_url }}/health"
 method: GET
 return_content: yes
 register: health_result
- name: Afficher le statut du site
 ansible.builtin.debug:
 msg: "Site {{ health_result.json.site }} : {{ health_result.json.status }}"
- name: Recuperer l'etat des interfaces du site
 ansible.builtin.uri:
 url: "{{ api_url }}/api/devices/{{ device_id_prefix }}-{{ inventory_hostname }}-01/interfaces"
 method: GET
 return_content: yes
 register: interfaces_result
- name: Identifier les interfaces en panne
 ansible.builtin.set_fact:
 interfaces_down: "{{ interfaces_result.json.interfaces | dict2items
 | selectattr('value.status', 'equalto', 'down') | map(attribute='key') | list }}"
- name: Alerter si des interfaces sont en panne
 ansible.builtin.debug:
 msg: "ATTENTION sur {{ inventory_hostname }} : interfaces en panne -> {{ interfaces_down }}"
 when: interfaces_down | length > 0

```
EOF
```

Étape 7 — Créer le playbook principal

```
cat > site.yml << 'EOF'
```

```
---
```

- name: Verification et configuration de base des sites NexaCorp
 hosts: all
 gather_facts: no
 roles:
 - base_config

E0F

Étape 8 — Démarrer le lab (TP4) et exécuter le playbook

```
cd lab
docker compose up -d --build
cd ..
ansible-playbook -i inventory/dynamic_inventory.py site.yml
```

Résultat attendu : les 3 sites (siege, agence1, agence2) passent en ok, et le dernier message debug affiche l’alerte sur l’interface en panne pour chacun.

Étape 9 — Cibler un groupe précis

```
ansible-playbook -i inventory/dynamic_inventory.py site.yml --limit hub
ansible-playbook -i inventory/dynamic_inventory.py site.yml --limit spokes
```

Étape 10 — Arrêter le lab, committer et pousser

```
cd lab && docker compose down && cd ..
git add inventory/ roles/ site.yml
git commit -m "TP5 : architecture Ansible modulaire avec roles et inventory dynamique"
git push -u origin <prenom>-<nom>-jour2-tp5
```

Ouvrir une pull request <prenom>-<nom>-jour2-tp5 → dev.

Comprendre le TP : comment Ansible s’articule

Cinq briques s’enchaînent à chaque exécution :

1. **L’inventaire** répond à “qui sont mes cibles ?”. Ici un inventaire *dynamique* (`dynamic_inventory.py`) génère la liste des hôtes à partir de `sites.yml`, rangés en groupes (hub, spokes) — ce qui permet de cibler juste une partie avec `--limit`.
2. **Le playbook** (`site.yml`) répond à “que faire, et sur qui ?” : il applique un rôle à un ensemble d’hôtes de l’inventaire.
3. **Le rôle** (`base_config`) est une recette réutilisable : une suite de tâches rangée dans une structure de dossiers standard (`tasks/`, `defaults/`), réutilisable sur d’autres sites ou d’autres projets sans réécrire la logique.
4. **Les modules** (ici `ansible.builtin.uri`) sont l’action concrète : chaque tâche appelle un module qui sait faire une chose précise — ici, envoyer une requête HTTP, l’équivalent Ansible du `requests.get()` du client Python du TP3, mais de façon déclarative.
5. **La cible réelle** est le lab Docker du TP4 : chaque appel `uri` contacte réellement `http://localhost:5001/5002/5003`.

Le réglage `ansible_connection: local` indique à Ansible de ne pas essayer de se connecter en SSH à ces “hôtes” (ce sont des APIs REST, pas des serveurs Linux) — tout s’exécute localement, et c’est le module `uri` qui parle réellement à la cible via HTTP.

Sur un vrai équipement Cisco, on utiliserait plutôt `network_cli` ou `httpapi` selon le type de connexion.

Validation

- ☐ `ansible --version` fonctionne
- ☐ `ansible-inventory -i inventory/dynamic_inventory.py --graph` affiche bien les groupes hub et spokes
- ☐ `ansible-playbook -i inventory/dynamic_inventory.py site.yml` s'exécute sans erreur sur les 3 sites (lab TP4 démarré au préalable)
- ☐ Le message d'alerte sur l'interface en panne s'affiche pour chaque site
- ☐ `--limit hub` et `--limit spokes` ciblent bien les bons sous-ensembles
- ☐ Une pull request <prenom>-<nom>-jour2-tp5 → dev est ouverte